Introduction to Style Transfer

You have seen that **Convolutional Neural Networks (CNNs)** are among the most powerful networks for image classification and analysis. CNNs process visual information in a feed-forward manner, passing an input image through a collection of image filters which extract certain features.

Beyond Classification

It turns out that these feature-level representations are not only useful for classification but also for **image construction**. These representations form the basis for creative applications such as:

- **Style Transfer**
- **Deep Dream**

These algorithms compose new images based on CNN layer activations and the features extracted during processing.

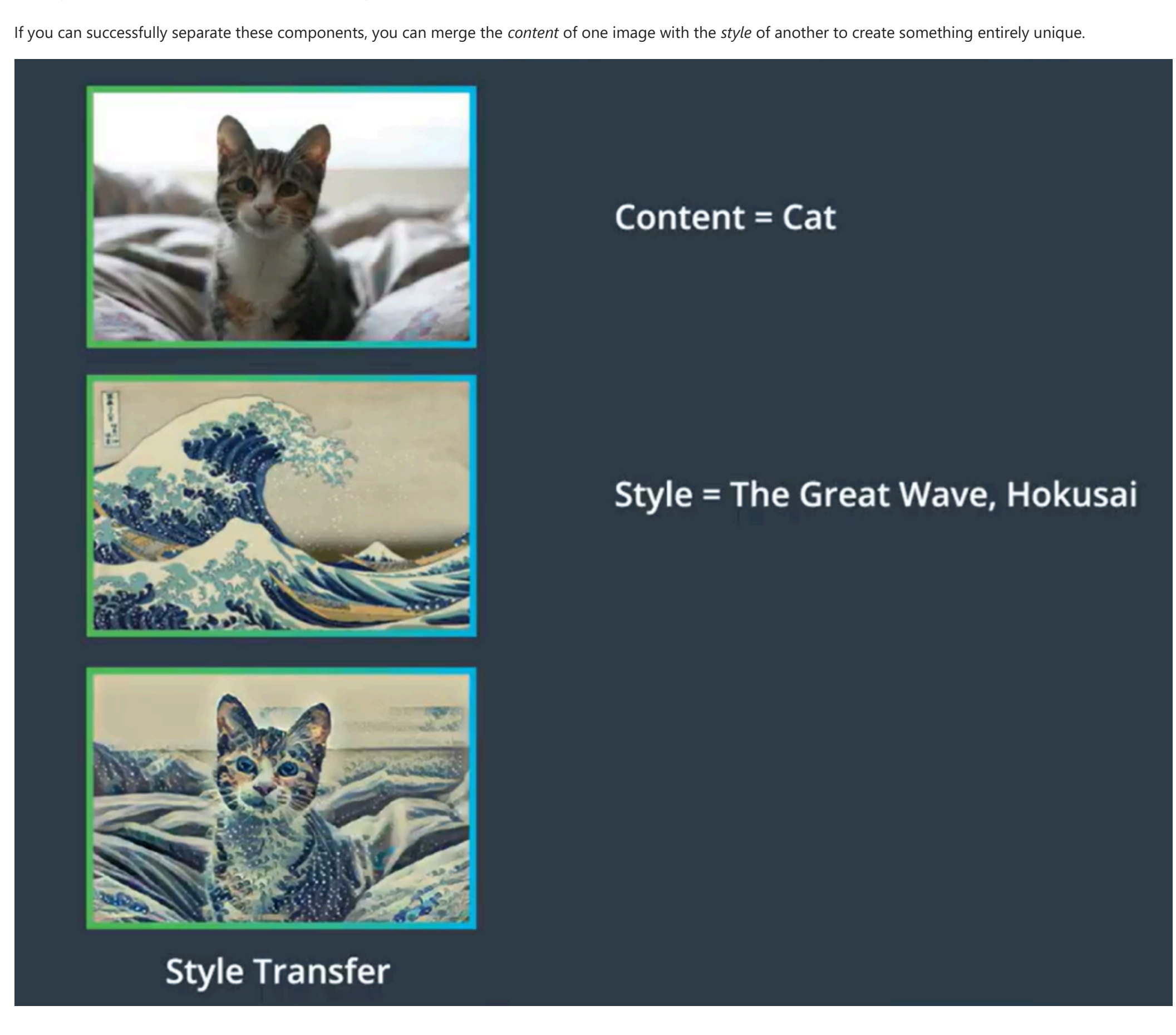
What is Style Transfer?

In this lesson, we will focus on learning about and implementing the **Style Transfer** algorithm. Style transfer allows you to apply the artistic style of one image to another image of your choice.

The key to this technique is using a trained CNN to **separate the content from the style** of an image:

- **Content**: The objects and arrangement in the image (e.g., a cat).
- **Style**: The artistic texture and color palette (e.g., Hokusai's *The Great Wave*).

If you can successfully separate these components, you can merge the *content* of one image with the *style* of another to create something entirely unique.


Content = Cat
Style = The Great Wave, Hokusai
Style Transfer

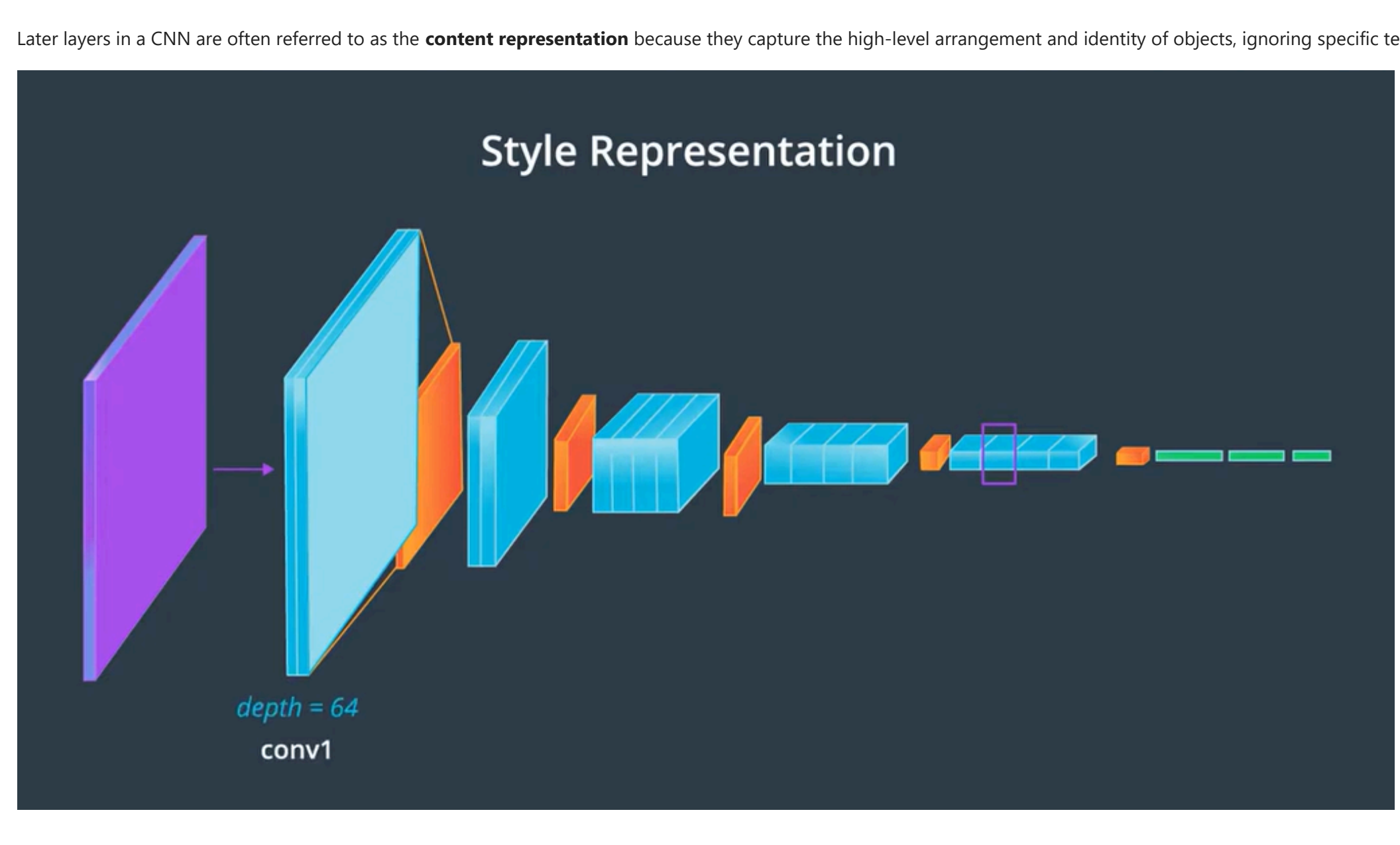
By the end of this lesson, you will have the knowledge needed to separate style and content to generate stylized images of your own design.

Separating Style & Content

When a CNN is trained for image classification, its layers learn to extract increasingly complex features. As we go deeper into the network, pooling layers discard detailed spatial information (like precise pixel locations) to focus on the semantic **content** of the image.

Content Representation

Later layers in a CNN are often referred to as the **content representation** because they capture the high-level arrangement and identity of objects, ignoring specific textures or colors.

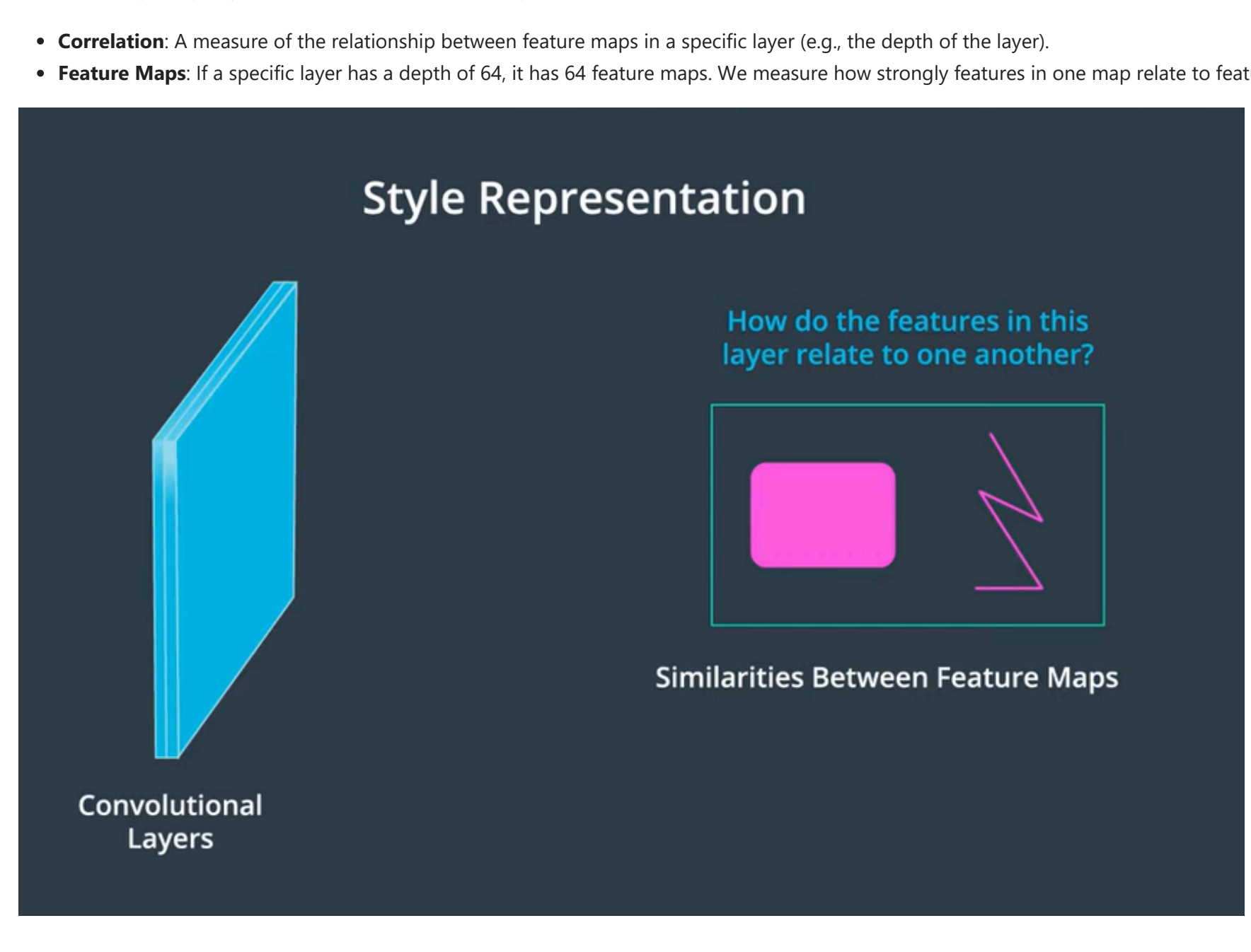

Style Representation
depth = 64
conv1

Style Representation

But what about style? **Style** refers to traits like brush strokes, texture, colors, and curvature. To separate style from content, we need to look at the "feature space" designed to capture texture and color information.

This is done by analyzing **spatial correlations** within a layer of the network.

- **Correlation**: A measure of the relationship between feature maps in a specific layer (e.g., the depth of the layer).
- **Feature Maps**: If a specific layer has a depth of 64, it has 64 feature maps. We measure how strongly features in one map relate to features in another.


Style Representation
How do the features in this layer relate to one another?
Similarities Between Feature Maps
Convolutional Layers

For example, asking "Is a certain color in map A always found with a certain edge in map B?" helps us derive the style. If there are common colors and shapes among feature maps, this collective information defines the **image's style**.

Combining Them

Style Transfer works by taking two images:

1. **Content Image**: Provides the objects and arrangement (from deep CNN layers).
2. **Style Image**: Provides the color, texture, and patterns (from correlations in feature maps).

The algorithm merges these to create a new third image that renders the original content using the artistic style of the second image.

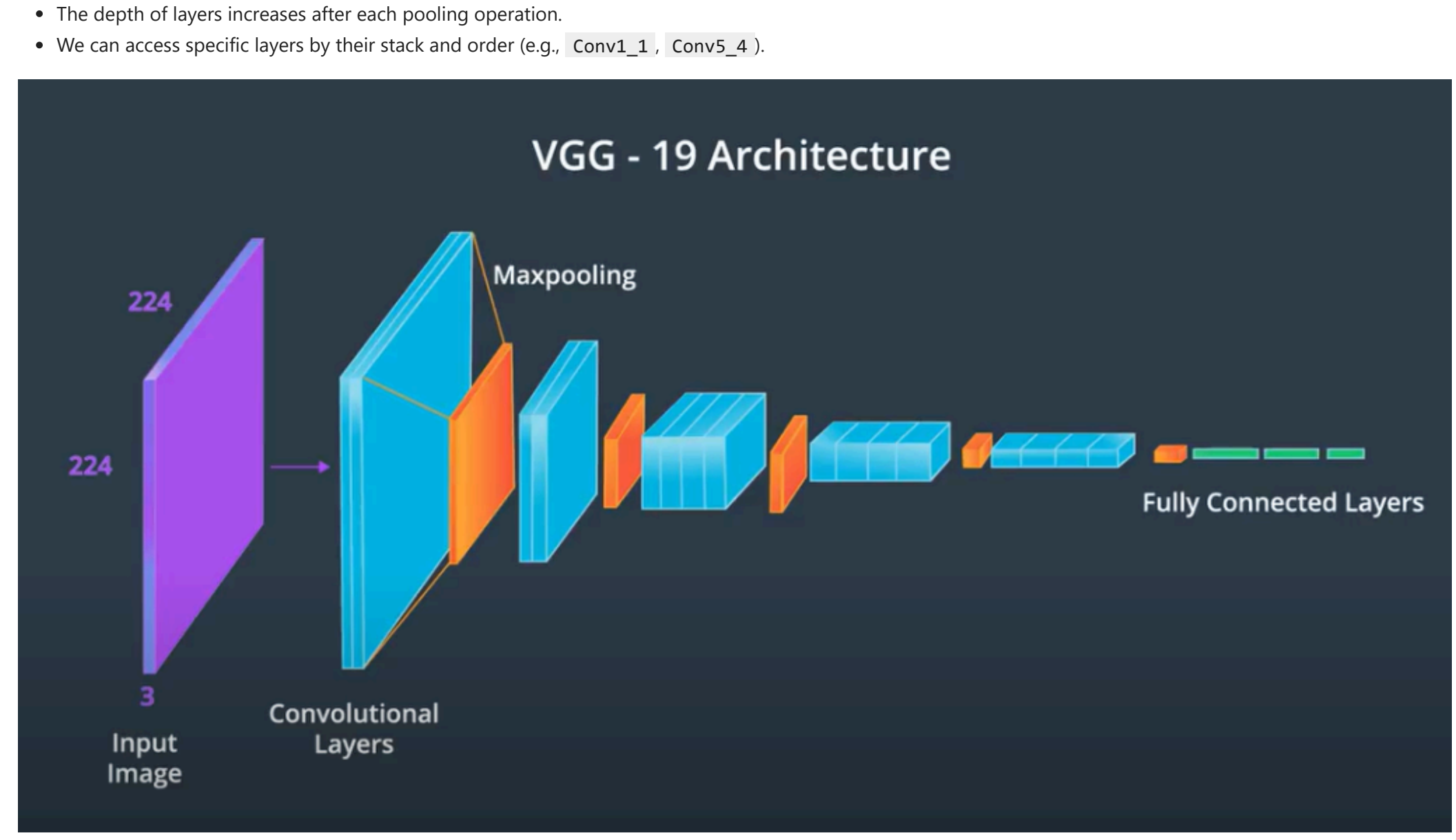
VGG19 & Content Loss

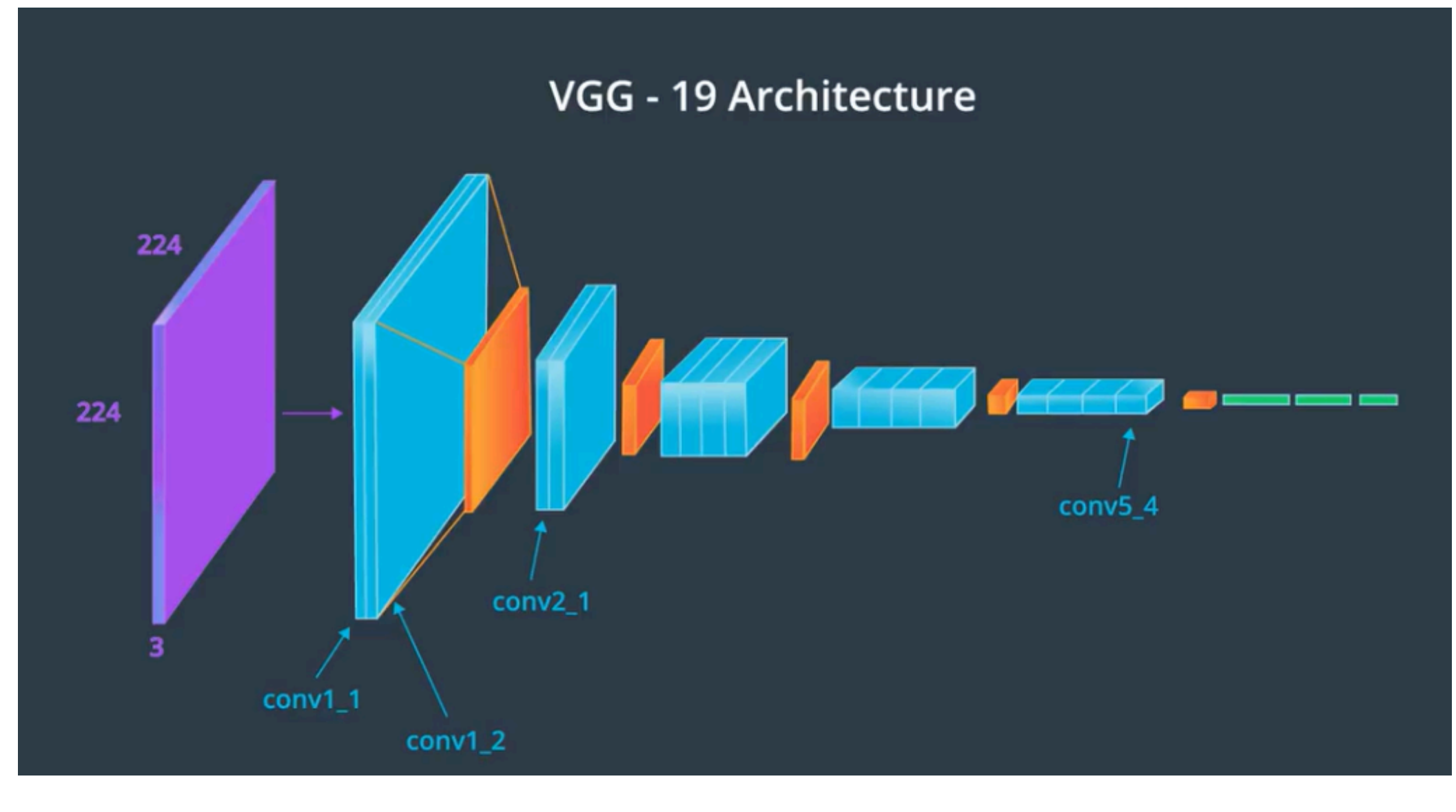
In this implementation, we follow the method outlined in the paper Image Style Transfer Using Convolutional Neural Networks by Gatys et al.

The VGG19 Network

We use a pre-trained **VGG19** network, a 19-layer CNN. We rely on it not for classification, but as a **feature extractor**.

- It consists of stacks of convolutional layers followed by pooling layers.
- The depth of layers increases after each pooling operation.
- We can access specific layers by their stack and order (e.g., `Conv1_1`, `Conv5_4`).


VGG - 19 Architecture
Maxpooling
224
224
3
Input Image
Convolutional Layers
Fully Connected Layers


VGG - 19 Architecture
224
224
3
conv1_1
conv1_2
conv2_1
conv5_4

The Goal

Both the **Content Image** and the **Style Image** are passed through VGG19.

- **Content Image** path: Extracts representations from deep layers (content).
- **Style Image** path: Extracts features from multiple layers (style).

We then create a **Target Image** (starting as a blank canvas or a copy of the content image) and iteratively update it so that:

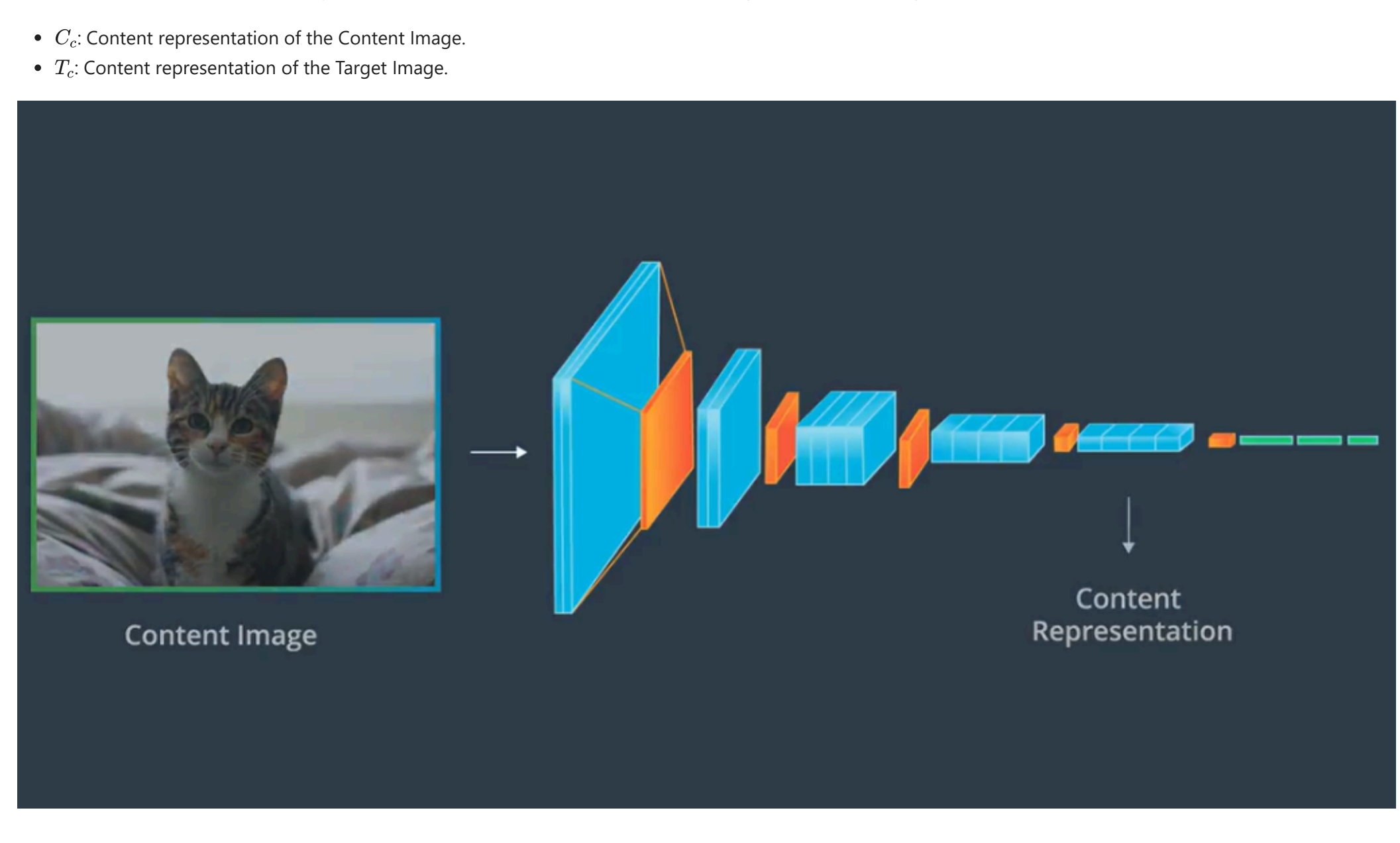
1. Its content matches the content image.
2. Its style matches the style image.

Content Loss

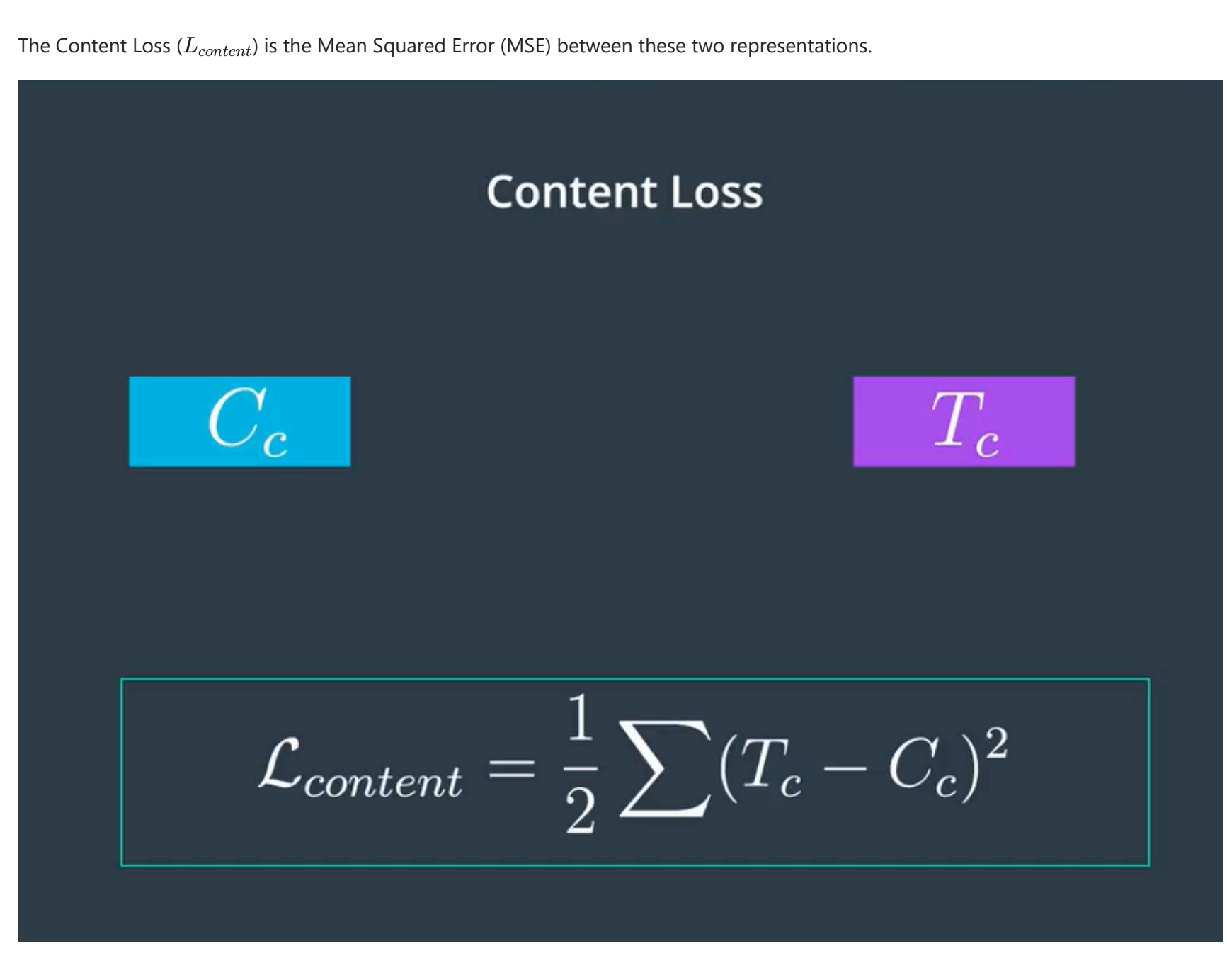
To make the target image look like the content image, we calculate the **Content Loss**.

We take the output from a specific layer (in the paper, **Conv4_2** is used) for both the target and content images.

- C_c: Content representation of the Content Image.
- T_c: Content representation of the Target Image.


Content Image
Content Representation

The Content Loss ($L_{content}$) is the Mean Squared Error (MSE) between these two representations.


Content Loss
C_c
T_c
$\mathcal{L}_{content} = \frac{1}{2}\sum(T_c - C_c)^2$

Our goal during the "training" (which actually updates the image pixels, not the network weights) is to **minimize this loss**. By doing so, the target image evolves to structurally resemble the content image.

Gram Matrix

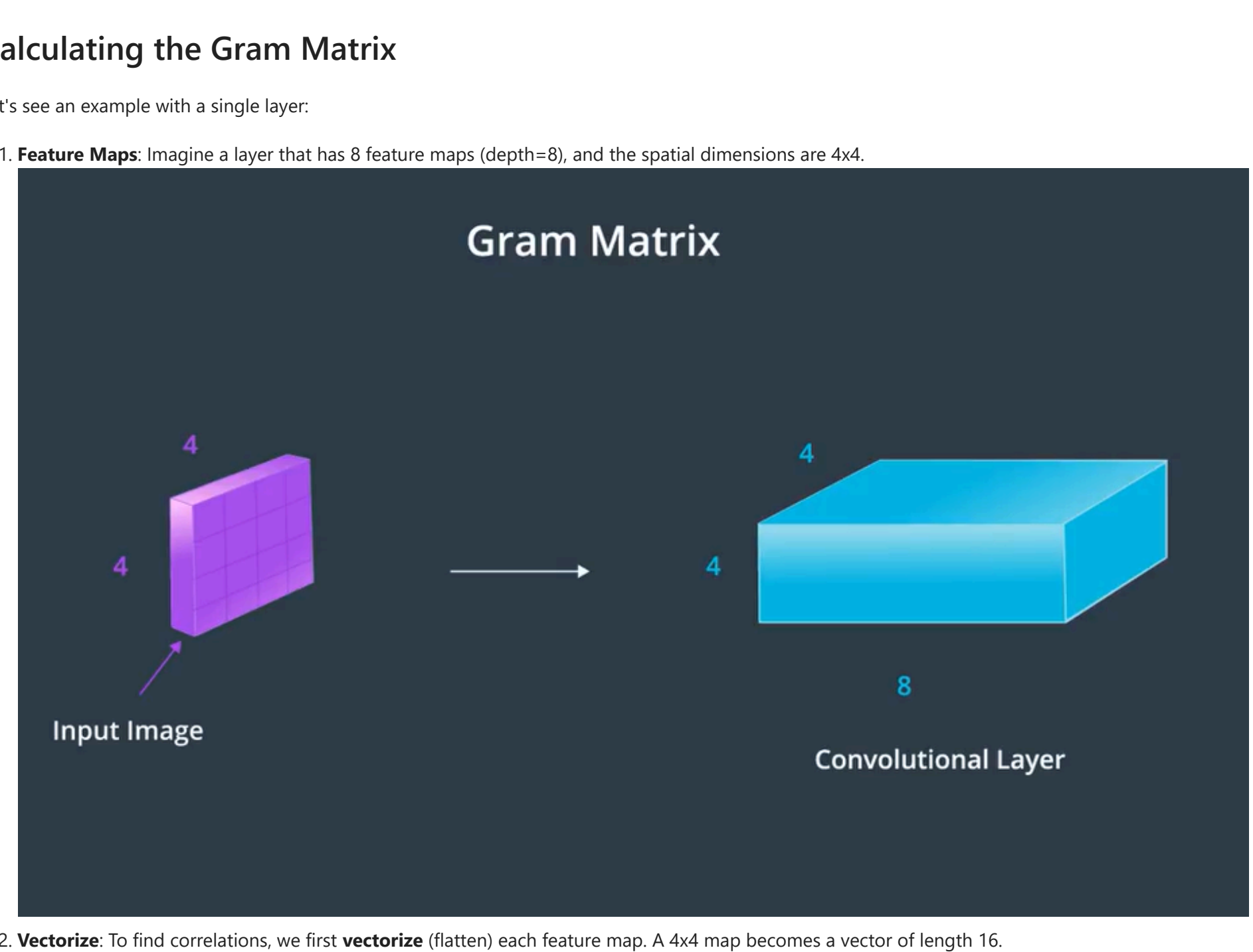
To capture **Style**, we look at correlations between features in the network's layers. We typically calculate this for multiple layers (from `conv1_1` up to `conv5_1`) to get a **multiscale style representation**.

The mathematical tool used to calculate these correlations is the **Gram Matrix**.

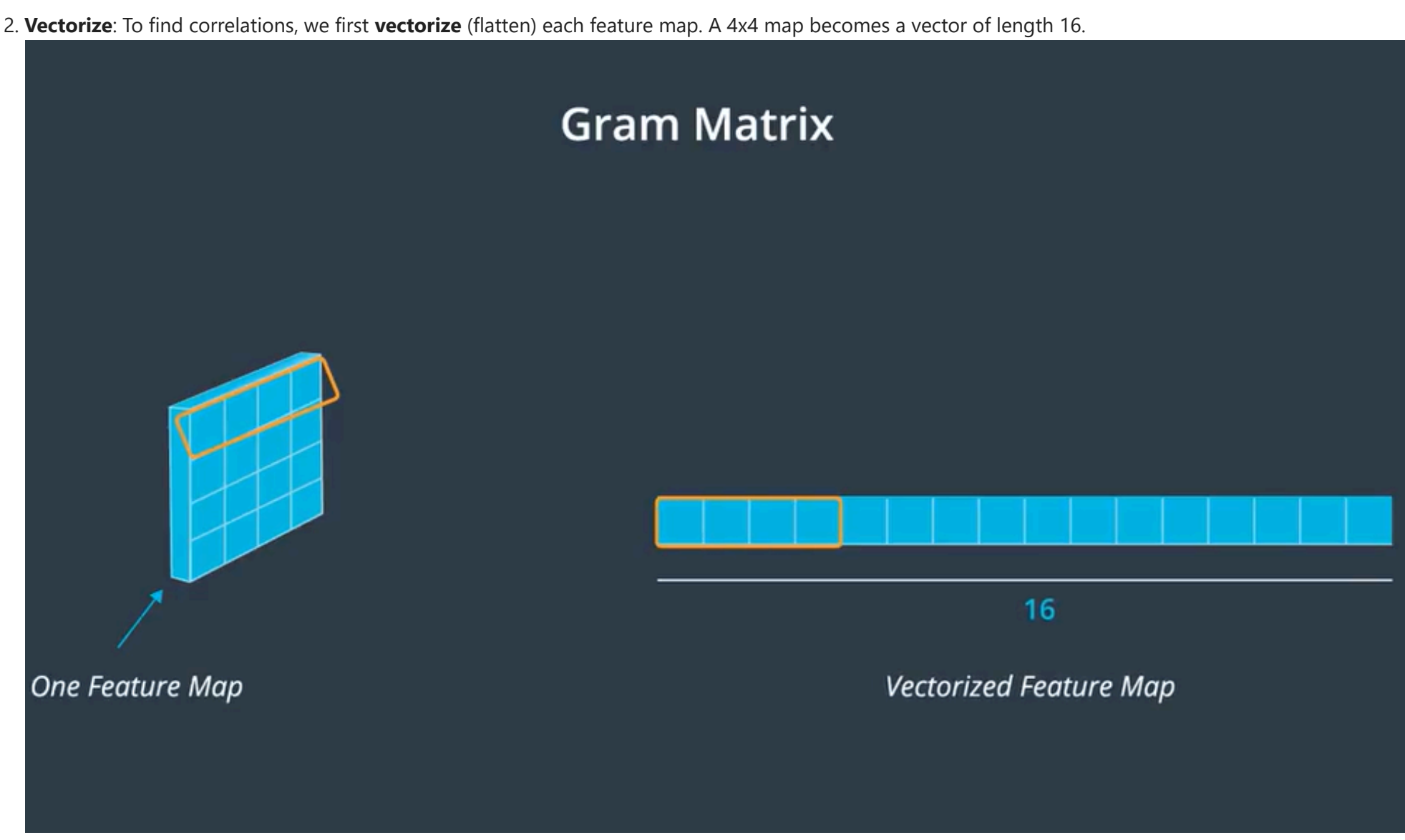
Calculating the Gram Matrix

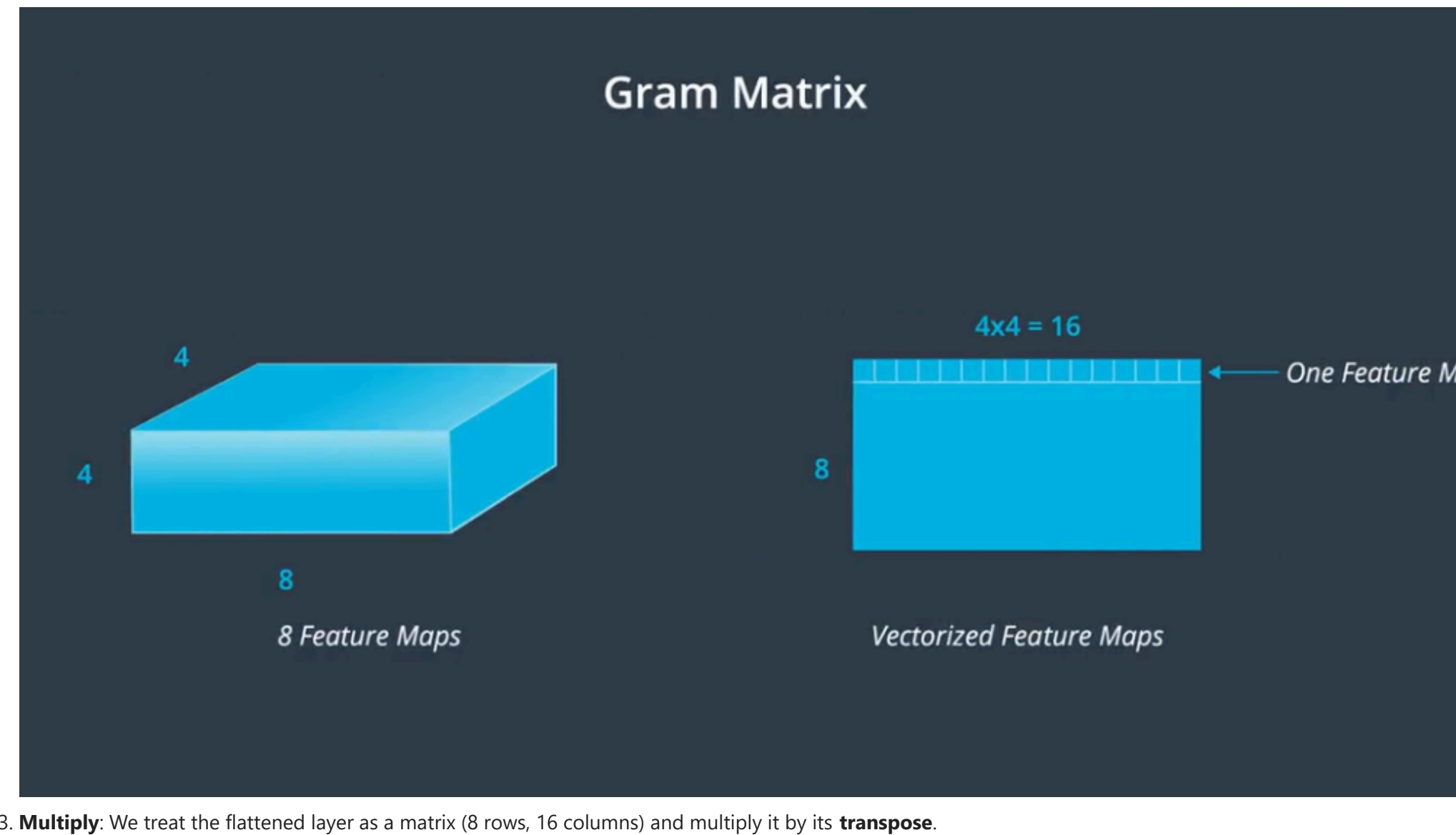
Let's see an example with a single layer:

1. **Feature Maps**: Imagine a layer that has 8 feature maps (depth=8), and the spatial dimensions are 4x4.

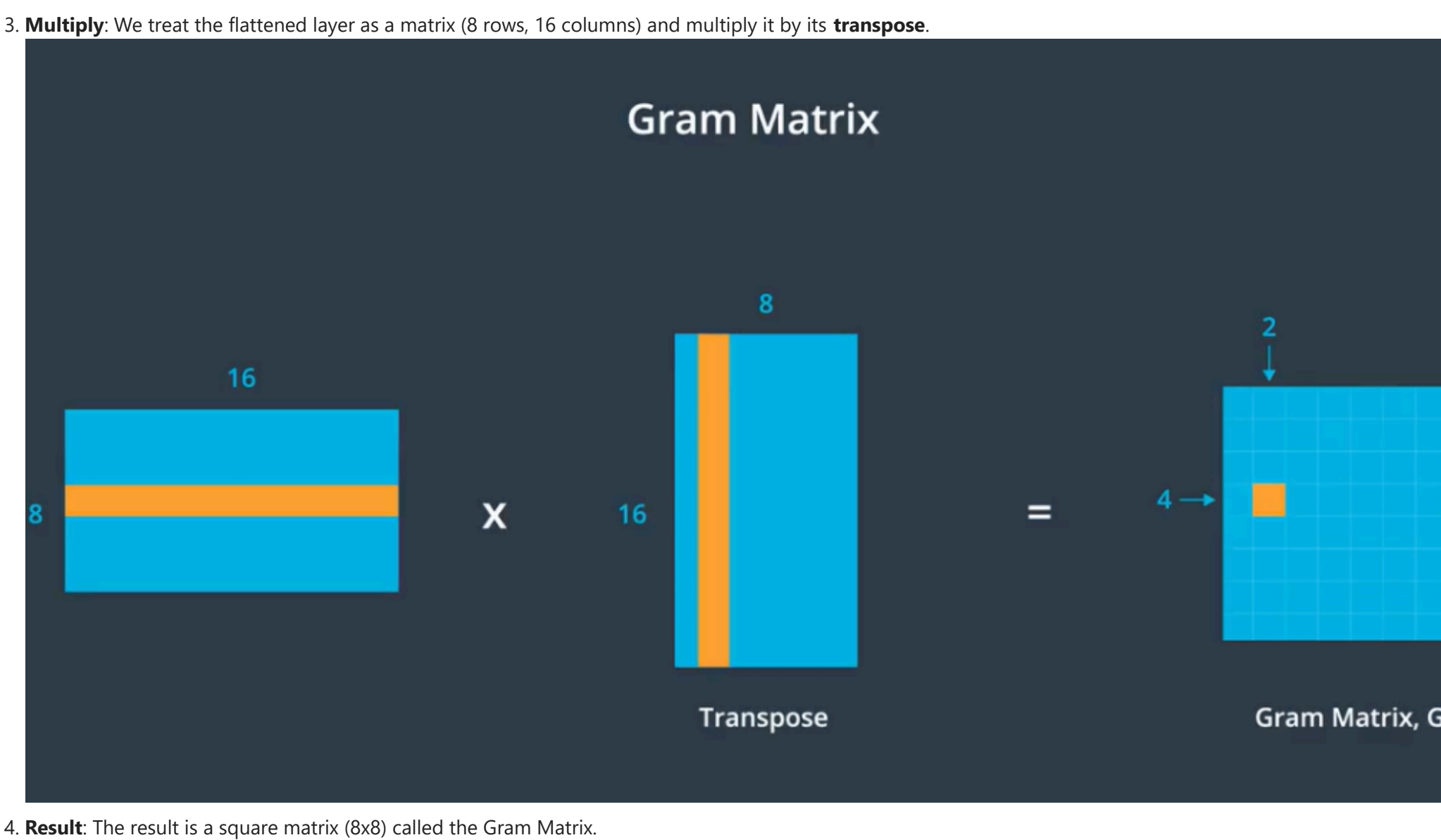

Gram Matrix
4
4
Input Image
4
4
8
Convolutional Layer

2. **Vectorize**: To find correlations, we first **vectorize** (flatten) each feature map. A 4x4 map becomes a vector of length 16.


Gram Matrix
One Feature Map
16
Vectorized Feature Map


Gram Matrix
4
4
8
8 Feature Maps
4x4 = 16
One Feature Map
8
Vectorized Feature Maps

3. **Multiply**: We treat the flattened layer as a matrix (8 rows, 16 columns) and multiply it by its **transpose**.


Gram Matrix
16
8
x
8
16
Transpose
=
2
4
Gram Matrix, G

4. **Result**: The result is a square matrix (8x8) called the Gram Matrix.

Why the Gram Matrix?

- **Non-localized Information**: By flattening the spatial dimensions (X, Y), we lose information about *where* things are in the image.
- **Style Texture**: What remains is information about *what* features typically appear together (e.g., texture, prominence of colors) regardless of position.

The values in the Gram Matrix (G) indicate similarities: $G_{4,2}$ tells us how similar the 4th and 2nd feature maps are. This effectively represents the "style" of that layer.

Style Loss

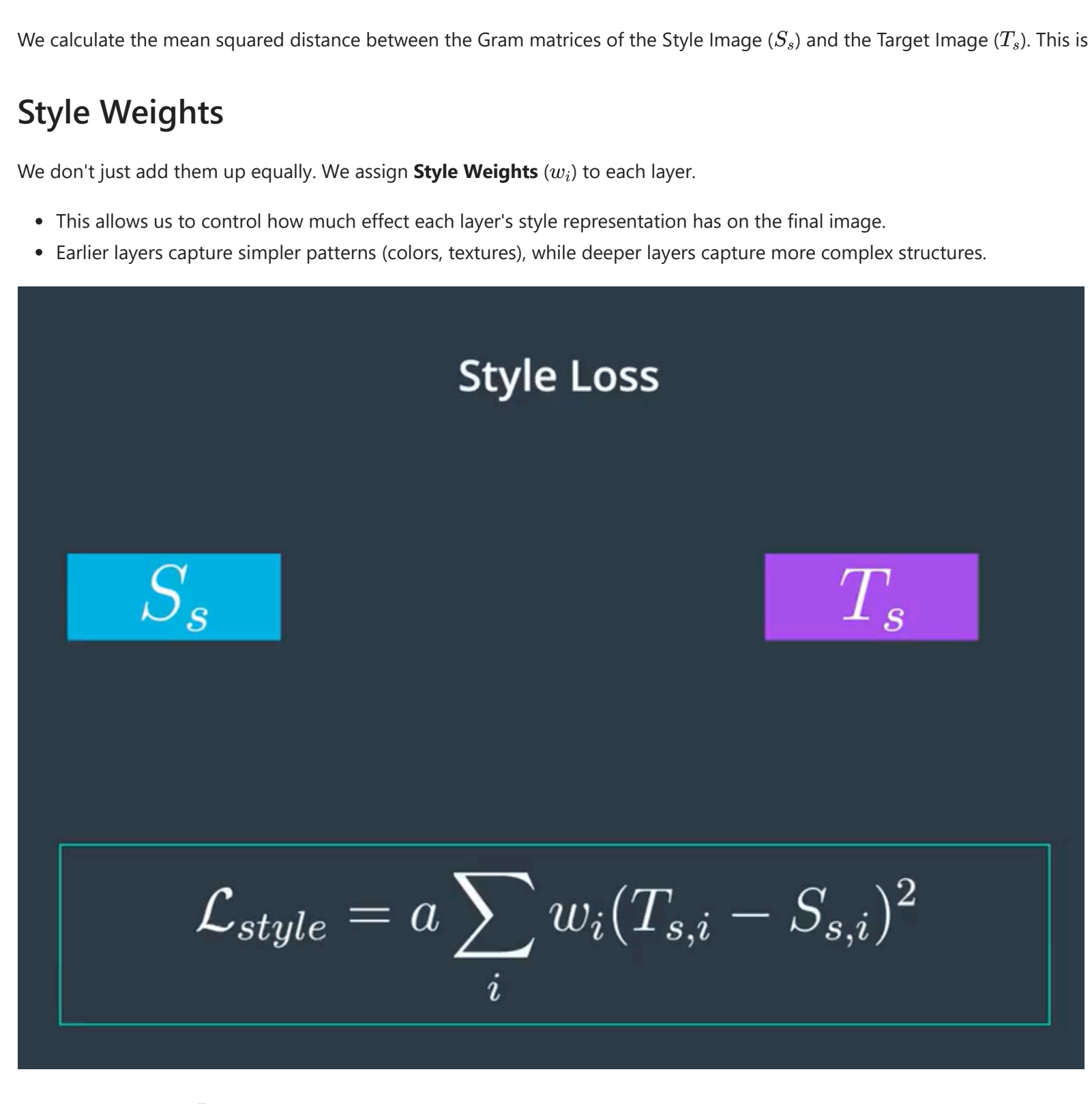
Once we have the Gram Matrices, we can calculate the **Style Loss**.

We calculate the mean squared distance between the Gram matrices of the Style Image (S_s) and the Target Image (T_s). This is done for all five layers (pairs) we selected (`conv1_1` through `conv5_1`).

Style Weights

We don't just add them up equally. We assign **Style Weights** (w_i) to each layer.

- This allows us to control how much effect each layer's style representation has on the final image.
- Earlier layers capture simpler patterns (colors, textures), while deeper layers capture more complex structures.


Style Loss
S_s
T_s
$\mathcal{L}_{style} = a\sum_i w_i(T_{s,i} - S_{s,i})^2$

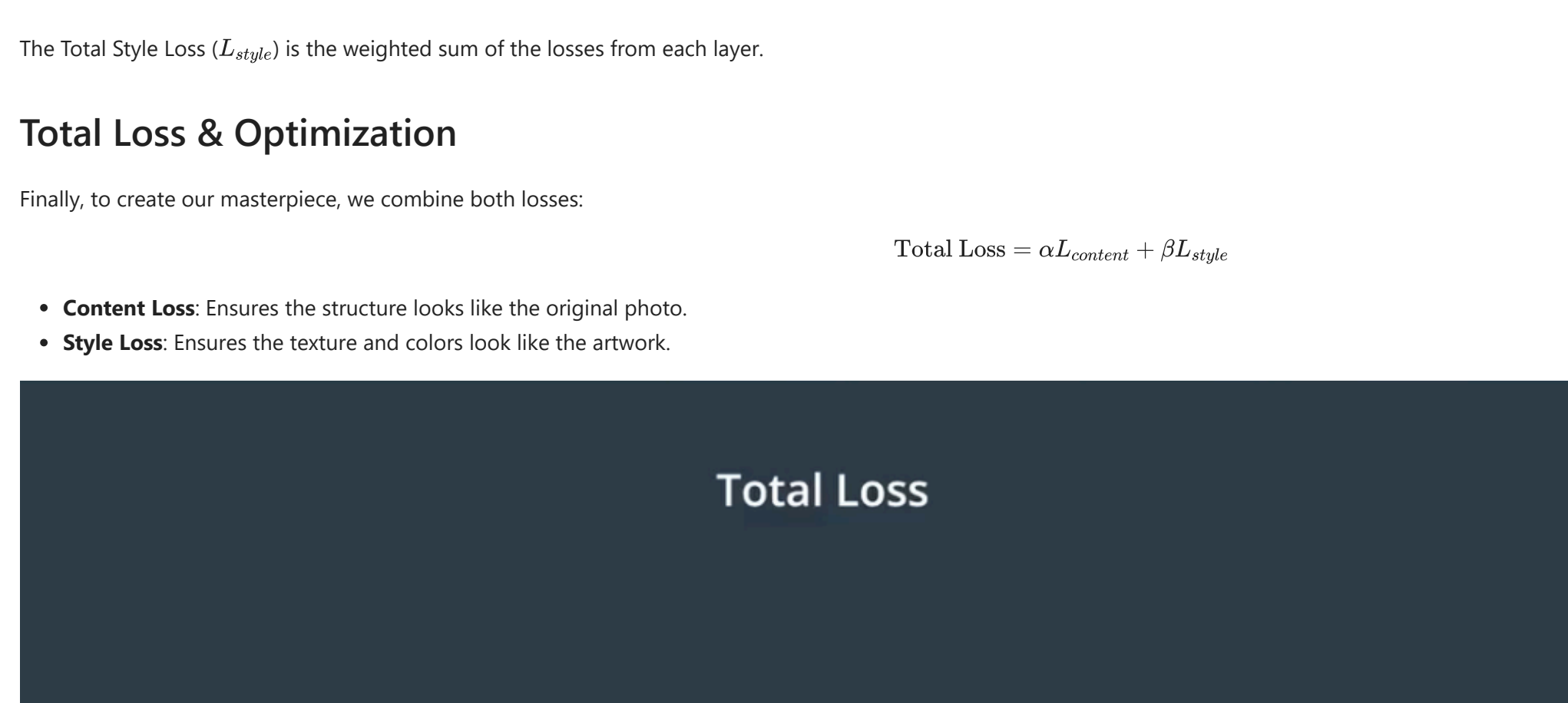
The Total Style Loss (L_{style}) is the weighted sum of the losses from each layer.

Total Loss & Optimization

Finally, to create our masterpiece, we combine both losses:

$$\text{Total Loss} = \alpha L_{content} + \beta L_{style}$$

- **Content Loss**: Ensures the structure looks like the original photo.
- **Style Loss**: Ensures the texture and colors look like the artwork.


Total Loss
$\mathcal{L}_{content} = \frac{1}{2}\sum(T_c - C_c)^2$
+
$\mathcal{L}_{style} = a\sum_i w_i(T_{s,i} - S_{s,i})^2$

We use standard **backpropagation** and optimization to minimize this total loss. However, instead of updating network weights, we iteratively **update the target image pixel values** until it matches our desired content and style.

Loss Weights

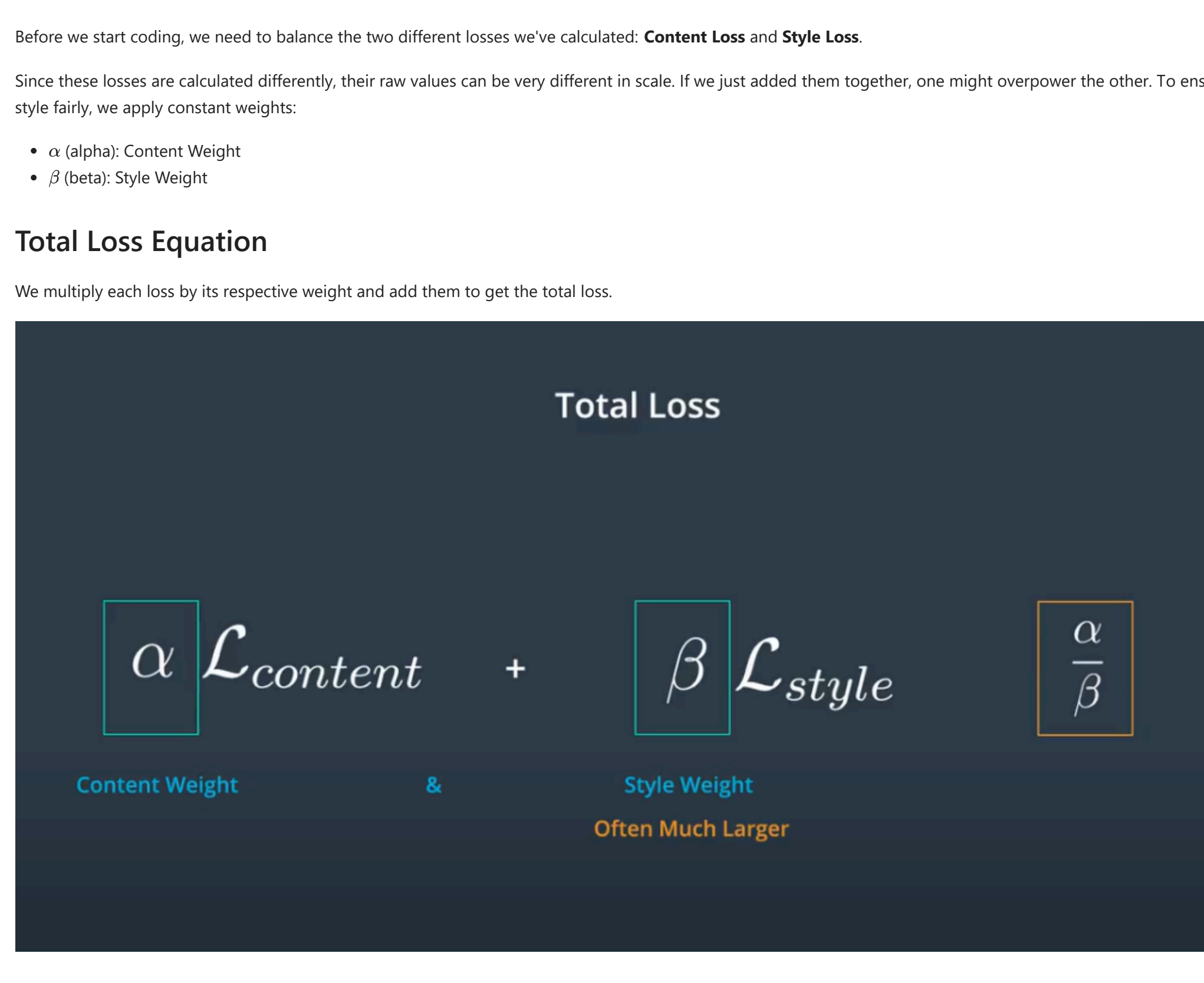
Before we start coding, we need to balance the two different losses we've calculated: **Content Loss** and **Style Loss**.

Since these losses are calculated differently, their raw values can be very different in scale. If we just added them together, one might overpower the other. To ensure our target image reflects both the content and style fairly, we apply constant weights:

- α (alpha): Content Weight
- β (beta): Style Weight

Total Loss Equation

We multiply each loss by its respective weight and add them to get the total loss.


Total Loss
$\alpha\,\mathcal{L}_{content}$
+
$\beta\,\mathcal{L}_{style}$
$\frac{\alpha}{\beta}$
Content Weight
&
Style Weight
Often Much Larger

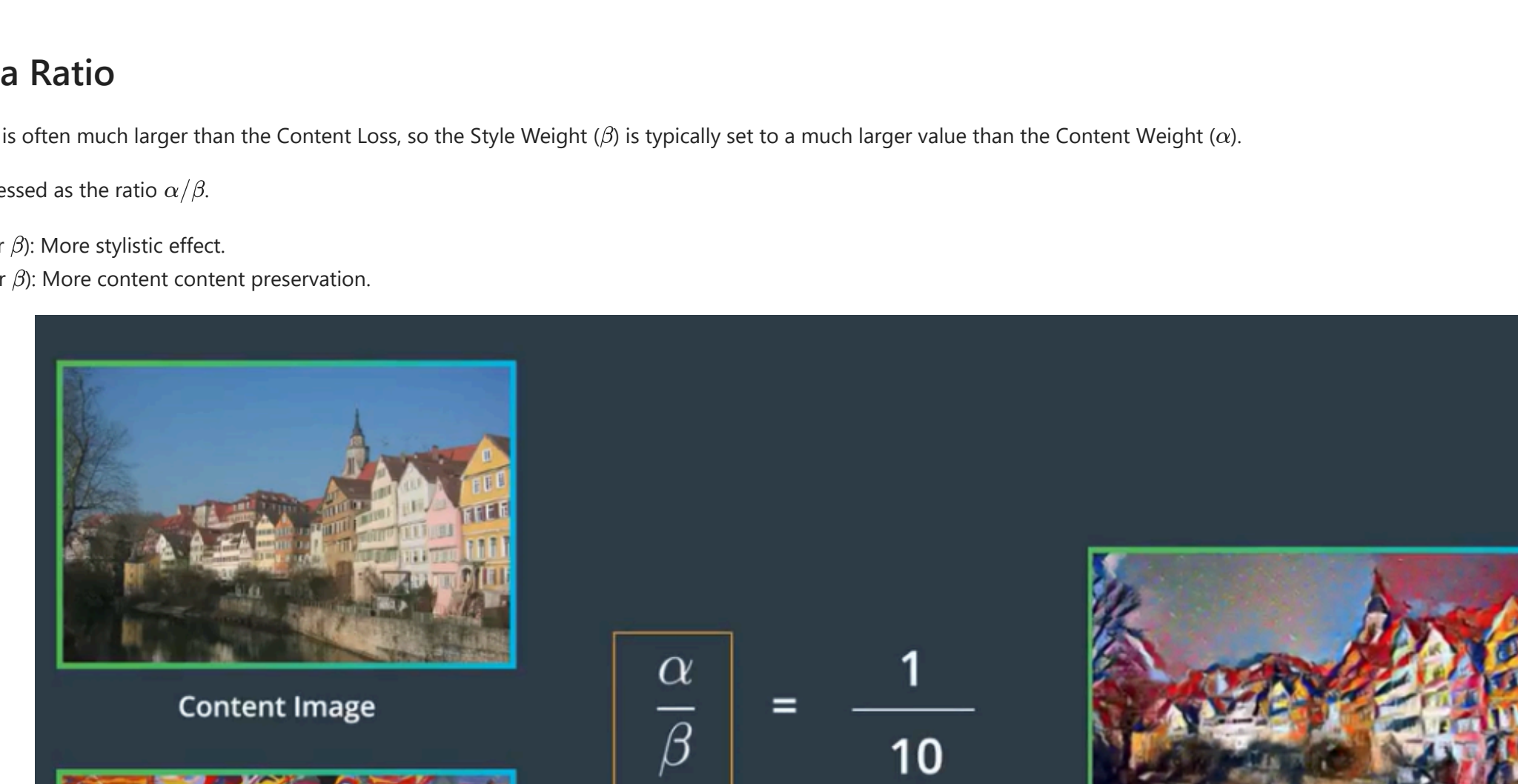
The Alpha/Beta Ratio

In practice, the Style Loss is often much larger than the Content Loss, so the Style Weight (β) is typically set to a much larger value than the Content Weight (α).

The balance is often expressed as the ratio α/β.

- **Smaller Ratio** (larger β): More stylistic effect.
- **Larger Ratio** (smaller β): More content content preservation.

For example, a ratio of $1/10$:

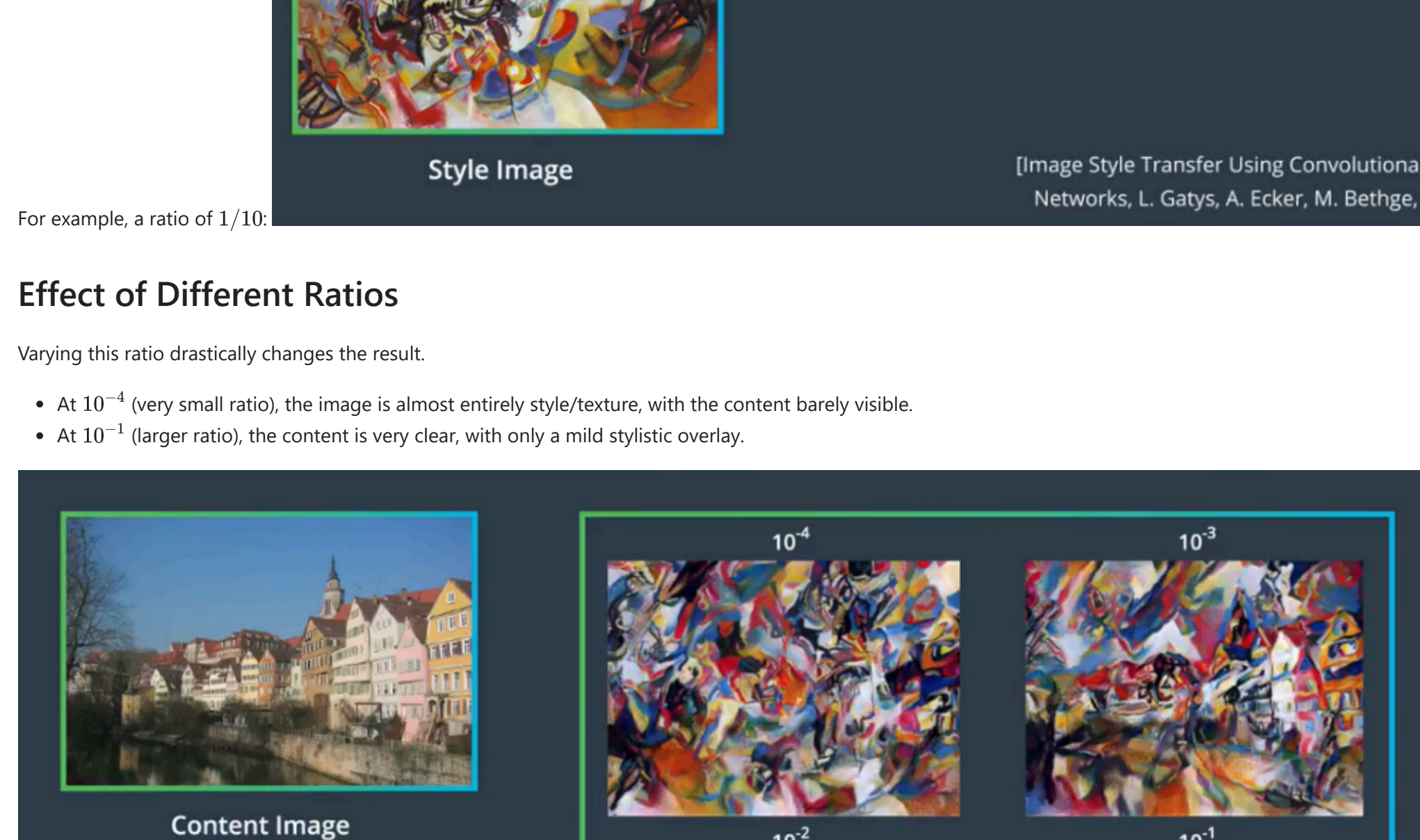

Content Image
Style Image
$\frac{\alpha}{\beta} = \frac{1}{10}$

[Image Style Transfer Using Convolutional Neural Networks, L. Gatys, A. Ecker, M. Bethge, 2016]

Effect of Different Ratios

Varying this ratio drastically changes the result.

- At 10^{-4} (very small ratio), the image is almost entirely style/texture, with the content barely visible.
- At 10^{-1} (larger ratio), the content is very clear, with only a mild stylistic overlay.


Content Image
Style Image
10^{-4}
10^{-3}
10^{-2}
10^{-1}

[Image Style Transfer Using Convolutional Neural Networks, L. Gatys, A. Ecker, M. Bethge, 2016]

These weights are hyperparameters you can tune to get the exact artistic effect you desire.